

Rapport de Projet : K-Partitionnement Robuste

MPRO 2025-2026

Bedis BACCAR & Rayen SELMI

Février 2026

Résumé

Ce document présente la modélisation et la résolution d'un problème de partitionnement de graphe sous incertitude (distances et poids). Nous avons formulé le problème sous forme d'un programme robuste Min-Max et implémenté trois méthodes exactes : la Dualisation (modèle monolithique), les Plans Coupants et le Branch-and-Cut. Conformément aux options du projet, nous avons également implémenté une heuristique (Choix 1) pour l'initialisation. Les résultats numériques montrent la supériorité de l'approche par Dualisation sur les instances testées.

Table des matières

1	Introduction	3
2	Modélisation du Problème Statique	3
3	Modélisation Robuste (Min-Max)	3
3.1	Objectif Robuste	3
3.2	Contraintes de Robustesse	3
4	Méthodes de Résolution Implémentées	4
4.1	Approche par Plans Coupants	4
4.1.1	Algorithme Maître-Esclave	4
4.1.2	Détails d'Implémentation	4
4.2	Approche Branch-and-Cut	4
4.2.1	Utilisation des Callbacks	5
4.3	Approche par Dualisation (Modèle Monolithique)	5
4.3.1	Transformation Mathématique	5
4.4	Heuristique Constructive (Option Choix 1)	5
4.4.1	Stratégie Least-Loaded Decreasing (LLD)	5
4.4.2	Analyse de la méthode	6
5	Stratégies d'Accélération et Optimisations	6
5.1	Brisure de Symétrie	6
5.2	Warm Start	7
5.3	Séparation Hybride : Heuristique et Exacte	7
6	Résultats Expérimentaux	7
6.1	Protocole Expérimental	7
6.2	Validation sur Petite Instance	8
6.3	Analyse Comparative par Régime de Difficulté	8
6.3.1	Régime 1 — Convergence exacte ($N \leq 22$)	8
6.3.2	Régime 2 — Zone de transition ($26 \leq N \leq 100$)	9
6.3.3	Régime 3 — Passage à l'échelle ($N > 200$)	9
6.4	Analyse du Prix de la Robustesse	10
6.5	Profil de Performance	11
7	Discussion et Recommandations	12
7.1	Choix de la Méthode Selon le Contexte	12
7.2	Pistes d'Amélioration	12
8	Conclusion	13
A	Résultats Complètes sur Toutes les Instances	14
B	Solutions Optimales (Échantillon)	14

1 Introduction

Le problème étudié consiste à partitionner les sommets d'un graphe $G = (V, E)$ en au plus K parties de capacité B , en minimisant le poids des arêtes internes. Dans sa version robuste, les distances et les poids sont soumis à des incertitudes bornées par des budgets L (distances) et W (poids).

Pour répondre aux objectifs du projet, nous avons adopté la stratégie suivante :

1. **Modélisation exacte** : Formulation PLNE compacte et reformulation robuste par dualité.
2. **Méthodes de résolution** :
 - **Dualisation** : Résolution d'un modèle unique intégrant les contraintes robustes.
 - **Plans Coupants** : Génération itérative de contraintes (Constraint Generation).
 - **Branch-and-Cut** : Intégration des coupes via des *Lazy Callbacks*.
3. **Option (Choix 1)** : Implémentation d'une **heuristique gloutonne** pour fournir une solution réalisable rapide (Warm Start).

2 Modélisation du Problème Statique

Le problème statique (sans incertitude) est modélisé comme suit. Soient $x_{ik} \in \{0, 1\}$ valant 1 si le sommet i est dans la partie k , et $y_{ij} \in \{0, 1\}$ valant 1 si l'arête (i, j) est interne.

$$\min \sum_{(i,j) \in E} l_{ij} y_{ij} \quad (1)$$

Sous contraintes :

$$\sum_{k=1}^K x_{ik} = 1 \quad \forall i \in V \quad (2)$$

$$\sum_{i \in V} w_i x_{ik} \leq B \quad \forall k \in \{1, \dots, K\} \quad (3)$$

$$y_{ij} \geq x_{ik} + x_{jk} - 1 \quad \forall (i, j) \in E, \forall k \quad (4)$$

3 Modélisation Robuste (Min-Max)

Nous considérons deux ensembles d'incertitude polyédraux U_1 (distances) et U_2 (poids). L'objectif est de minimiser le pire coût possible tout en garantissant la faisabilité dans le pire scénario de poids.

3.1 Objectif Robuste

$$\min_{x,y} \max_{\delta^1 \in U_1} \sum_{(i,j) \in E} (l_{ij} + \delta_{ij}^1 (\hat{l}_i + \hat{l}_j)) y_{ij} \quad (5)$$

Avec $U_1 = \{\delta^1 \in [0, 3]^{|E|} \mid \sum \delta_{ij}^1 \leq L\}$.

3.2 Contraintes de Robustesse

Pour chaque partie k , la capacité doit être respectée dans le pire cas :

$$\max_{\delta^2 \in U_2} \sum_{i \in V} w_i (1 + \delta_i^2) x_{ik} \leq B, \quad \forall k \quad (6)$$

Avec $U_2 = \{\delta^2 \in [0, W_v]^{|V|} \mid \sum \delta_i^2 \leq W\}$.

4 Méthodes de Résolution Implémentées

4.1 Approche par Plans Coupants

Cette méthode décompose le problème en un **Problème Maître Restreint (PMR)** et des sous-problèmes de séparation. Le PMR est une relaxation du problème robuste contenant un nombre limité de contraintes.

4.1.1 Algorithme Maître-Esclave

À chaque itération, nous résolvons le PMR pour obtenir une solution candidate (x^*, y^*, z^*) . Nous vérifions ensuite si cette solution respecte les contraintes de robustesse en résolvant les sous-problèmes de séparation (SP1 pour l'objectif, SP2 pour la faisabilité).

Algorithm 1 Plans Coupants Robustes

```
1: Initialisation :  $U_1^* = \{0\}$ ,  $U_2^* = \{0\}$  (scénario nominal uniquement)
2: Construire le PMR avec les contraintes statiques
3: repeat
4:   Résoudre le PMR  $\rightarrow$  Solution  $(x^*, y^*, z^*)$ 
5:    $Cuts\_Added \leftarrow \text{False}$ 
6:   // Séparation SP1 (Objectif)
7:   Résoudre  $\max_{\delta^1 \in U_1} \sum_{ij} (\hat{\ell}_i + \hat{\ell}_j) y_{ij}^* \delta_{ij}^1$  pour obtenir le pire scénario  $\delta^{1*}$ 
8:   if  $\sum_{ij} l_{ij} y_{ij}^* + \text{Coût}(\delta^{1*}) > z^* + \epsilon$  then
9:     Ajouter la coupe :  $z \geq \sum_{ij} l_{ij} y_{ij} + \sum_{ij} (\hat{\ell}_i + \hat{\ell}_j) y_{ij} \delta_{ij}^{1*}$ 
10:     $Cuts\_Added \leftarrow \text{True}$ 
11:   end if
12:   // Séparation SP2 (Faisabilité)
13:   for  $k = 1, \dots, K$  do
14:     Résoudre  $\max_{\delta^2 \in U_2} \sum_i w_i x_{ik}^* (1 + \delta_i^2)$  pour obtenir  $\delta^{2*}$ 
15:     if  $\text{Poids\_Pire\_Cas}(k) > B + \epsilon$  then
16:       Ajouter la coupe :  $\sum_i w_i x_{ik} (1 + \delta_i^{2*}) \leq B$ 
17:        $Cuts\_Added \leftarrow \text{True}$ 
18:     end if
19:   end for
20: until Aucune coupe ajoutée ou Timeout atteint
```

4.1.2 Détails d'Implémentation

Pour optimiser la résolution du PMR à chaque itération, nous avons appliqué une relaxation sur les variables y_{ij} . Dans le code Julia, elles sont déclarées continues : `@variable(model, 0 <= y[e in E] <= 1)`. Puisque l'objectif minimise les coûts positifs et que les contraintes de liaison $y_{ij} \geq x_{ik} + x_{jk} - 1$ forcent y à 1 si les sommets sont dans la même partition, y prendra naturellement des valeurs entières à l'optimalité. Cela réduit considérablement la complexité du problème maître.

4.2 Approche Branch-and-Cut

L'inconvénient des plans coupants classiques est la nécessité de relancer l'optimisation complète à chaque itération. Le Branch-and-Cut intègre la génération de coupes directement dans l'arbre de recherche du solveur (Gurobi).

4.2.1 Utilisation des Callbacks

Nous utilisons deux types de callbacks pour intervenir à différents niveaux de l'arbre :

- **Lazy Constraint Callback** : Appelé lorsqu'une solution **entière** est trouvée. Nous vérifions la robustesse (séparation SP1 et SP2). Si la solution n'est pas robuste, on ajoute la coupe correspondante ("Lazy Constraint") pour invalider cette solution entière. C'est essentiel pour la correction de l'algorithme.
- **User Cut Callback** : Appelé sur les solutions **fractionnaires** (nœuds de l'arbre). Nous tentons de séparer des coupes basées sur la relaxation linéaire. Bien que non obligatoire pour la validité, cela permet de renforcer la relaxation et de couper des branches de l'arbre plus tôt, améliorant ainsi la performance globale.

4.3 Approche par Dualisation (Modèle Monolithique)

Cette méthode consiste à reformuler le problème robuste Min-Max en un unique problème de minimisation (PLNE compact) en utilisant la dualité forte de la programmation linéaire.

4.3.1 Transformation Mathématique

Pour chaque contrainte robuste (qui contient une maximisation sur l'incertitude), nous remplaçons le sous-problème "pire cas" par son dual de minimisation. Nous introduisons les variables duales :

- λ, μ_{ij} pour l'incertitude sur l'objectif (distances).
- α_k, β_{ik} pour l'incertitude sur la faisabilité (poids).

Le modèle robuste complet devient :

$$\min \sum_{(i,j) \in E} l_{ij} y_{ij} + L\lambda + \sum_{(i,j) \in E} 3\mu_{ij} \quad (7)$$

Sous contraintes, en plus des Contraintes du modèle statique :

$$\lambda + \mu_{ij} \geq (\hat{l}_i + \hat{l}_j) y_{ij} \quad \forall (i, j) \in E \quad (8)$$

$$\sum_{i \in V} w_i x_{ik} + W\alpha_k + \sum_{i \in V} W_v \beta_{ik} \leq B \quad \forall k \quad (9)$$

$$\alpha_k + \beta_{ik} \geq w_i x_{ik} \quad \forall i, k \quad (10)$$

4.4 Heuristique Constructive (Option Choix 1)

L'heuristique gloutonne développée est axée sur la rapidité destinée à fournir une solution initiale admissible aux solveurs exacts (*Warm Start*). Étant donné la difficulté de satisfaire les contraintes de capacité robustes tout en minimisant la coupe, notre approche simplifie temporairement le problème en le considérant comme un problème de *Bin Packing* pur. Nous ignorons la topologie du graphe (les coûts des arêtes) pour nous concentrer exclusivement sur la satisfaction des contraintes de poids.

4.4.1 Stratégie Least-Loaded Decreasing (LLD)

Nous avons opté pour une stratégie gloutonne de type **Least-Loaded Decreasing**. Cette méthode privilégie l'équilibrage des charges entre les K partitions, ce qui maximise les chances d'insérer tous les sommets sans violer la capacité B .

L'algorithme procède en deux temps :

1. **Tri** : Les sommets sont triés par ordre décroissant de poids nominal (w_i). Les éléments les plus lourds, plus difficiles à placer, sont traités en premier.

2. **Affectation** : Chaque sommet est placé dans la partition admissible qui possède la charge courante la plus faible.

Algorithm 2 Heuristique de Partitionnement (Approche Bin Packing)

```

1: Entrée : Sommets  $V$ , Poids  $w$ , Capacité  $B$ , Nombre de partitions  $K$ 
2: Sortie : Partitionnement  $\mathcal{P} = \{P_1, \dots, P_K\}$ 
3: Trier les sommets tel que  $w_{i_1} \geq w_{i_2} \geq \dots \geq w_{i_n}$ 
4: Initialiser  $W_k \leftarrow 0$  pour tout  $k \in \{1, \dots, K\}$ 
5: for  $j = 1$  to  $n$  do
6:   Soit  $i = i_j$  le sommet courant
7:   Identifier les partitions valides :  $\mathcal{K}_{valide} = \{k \mid W_k + w_i \leq B\}$ 
8:   if  $\mathcal{K}_{valide} \neq \emptyset$  then
9:     Choix LLD : Sélectionner  $k^* = \arg \min_{k \in \mathcal{K}_{valide}} W_k$ 
10:    Ajouter  $i$  à la partition  $P_{k^*}$ 
11:    Mettre à jour la charge :  $W_{k^*} \leftarrow W_{k^*} + w_i$ 
12:   else
13:     Arrêt : Échec de l'heuristique (instance non réalisable par cette méthode)
14:   end if
15: end for
16: return Solution  $\mathcal{P}$ 

```

4.4.2 Analyse de la méthode

Cette heuristique joue un rôle utilitaire précis dans notre projet :

- **Rapidité et Faisabilité** : La complexité est dominée par le tri ($O(N \log N)$). Sur nos instances de test, le temps d'exécution est négligeable (< 1 ms pour $N = 200$), ce qui permet de l'exécuter systématiquement avant le lancement de Gurobi.
- **Délégation de l'optimisation** : L'heuristique ignore volontairement la minimisation des arêtes internes. Ce choix est justifié par notre architecture de résolution : l'heuristique se charge uniquement de trouver une solution admissible, laissant la tâche complexe de minimiser l'objectif aux méthodes exactes (Dualisation, Branch-and-Cut) qui sont conçues pour explorer l'espace de recherche en profondeur.
- **Rôle de Warm Start** : Bien que la qualité de l'objectif initial soit modeste, fournir une solution entière réalisable dès le nœud racine est crucial. Cela permet au solveur d'initialiser une borne supérieure valide (Upper Bound) et d'activer immédiatement les mécanismes d'élagage de l'arbre, accélérant ainsi la convergence des méthodes exactes.

5 Stratégies d'Accélération et Optimisations

La complexité NP-difficile du problème de partitionnement, combinée à la lourdeur des contraintes robustes, entraîne une explosion combinatoire rapide. Pour traiter les instances de taille moyenne ($N > 40$), nous avons enrichi les algorithmes de base avec trois couches d'optimisation technique.

5.1 Brisure de Symétrie

Le problème de partitionnement souffre d'une symétrie inhérente : toute permutation des indices des partitions $k \in \{1, \dots, K\}$ produit une solution mathématiquement distincte mais

physiquement identique (même coût, mêmes groupes). Cette redondance multiplie l'espace de recherche par $K!$, ralentissant considérablement l'exploration de l'arbre de Branch-and-Bound.

Pour canoniser la solution, nous imposons un **ordre strict** sur l'utilisation des partitions :

1. **Ancrage** : Le premier sommet est forcé d'appartenir à la première partition.

$$x_{1,1} = 1 \quad (11)$$

2. **Ordre Séquentiel** : La partition $k + 1$ ne peut être "ouverte" (contenir des sommets) que si la partition k contient déjà au moins un sommet d'indice inférieur.

$$\sum_{j=1}^i x_{j,k} \geq x_{i,k+1} \quad \forall i \in \{2, \dots, n\}, \forall k \in \{1, \dots, K-1\} \quad (12)$$

Cette contrainte permet au solveur d'élaguer (prune) toutes les branches symétriques isomorphes, divisant l'espace de recherche effectif.

5.2 Warm Start

Les solveurs MIP comme Gurobi sont très sensibles à la qualité de la borne supérieure initiale (Primal Bound). Une bonne solution initiale permet d'élaguer les nœuds de l'arbre dont l'évaluation est inférieure à cette borne.

Nous injectons la solution trouvée par l'**Heuristique Gloutonne** (Choix 1) directement dans le modèle exact :

- Les variables x_{ik} sont initialisées via `set_start_value` avec la configuration heuristique.
- **Impact** : Le solveur commence l'optimisation avec un Gap Primal-Dual réduit, évitant la phase coûteuse de recherche de la première solution entière réalisable.

5.3 Séparation Hybride : Heuristique et Exacte

Dans les méthodes de Plans Coupants et Branch-and-Cut, le goulot d'étranglement est la résolution répétée des sous-problèmes de séparation (SP1 et SP2). Appeler un solveur LP à chaque callback est prohibitif.

Nous avons implémenté une stratégie de séparation à deux niveaux :

1. **Niveau 1 (Algorithmique)** : Les sous-problèmes robustes (incertitude budgétée) s'apparentent à des problèmes du *sac-à-dos continu*. Nous les résolvons par un algorithme de tri glouton en $O(E \log E)$:
 - Pour SP1, on trie les arêtes par impact décroissant $(\hat{\ell}_i + \hat{\ell}_j)y_{ij}^*$.
 - On sature le budget d'incertitude L sur les éléments les plus impactants.
2. **Niveau 2 (Exact)** : Uniquement si l'algorithme glouton ne trouve pas de violation (ou si la solution est complexe), nous appelons le solveur exact Gurobi pour garantir la validité mathématique (Correction).

Cette approche hybride permet de générer 99% des coupes en une fraction de seconde, réservant le solveur lourd aux cas limites.

6 Résultats Expérimentaux

6.1 Protocole Expérimental

Le temps limite est fixé à 80s pour le benchmark principal et étendu à 150s pour des analyses ciblées. Chaque instance est résolue par les quatre méthodes : Dualisation, Plans Coupants, Branch-and-Cut, et Heuristique gloutonne. Les tableaux complets figurent en Annexe A.

6.2 Validation sur Petite Instance

Afin de vérifier la correction des algorithmes, nous avons d’abord exécuté les quatre méthodes sur l’instance 10_ulysses_3 ($n = 10$, $K = 3$), pour laquelle l’optimum robuste est connu.

TABLE 1 – Validation sur 10_ulysses_3 : convergence vers l’optimum

Méthode	Objectif	Temps (s)	Optimal ?
Statique	54.36	0.10	✓
Dualisation	137.00	0.24	✓
Plans Coupants	137.00	1.40	✓
Branch-and-Cut	137.00	1.98	✓
Heuristique	354.51	0.01	–

Les trois méthodes exactes convergent vers l’objectif robuste de **137.00**, identique à la solution théorique de référence. La partition retournée par la Dualisation ($P_1 = \{1, 2, 3, 10\}$, $P_2 = \{4, 6, 7, 8\}$, $P_3 = \{5, 9\}$) coïncide exactement avec la solution connue, validant la modélisation des contraintes de capacité. Le prix de la robustesse sur cette instance est de +152%, signe que la solution statique est très vulnérable aux incertitudes paramétrées par Γ et Λ .

```

=====
                                TABLEAU COMPARATIF FINAL
=====
Méthode | Obj. Value | Temps (s) | Gap (%) | Statut | Amélioration | PR (%)
-----|-----|-----|-----|-----|-----|-----
Statique | 54.3548 | 0.67 | 0.0000 | ✓ Optimal | - | -
Dualisation | 136.9953 | 0.24 | 0.0000 | ✓ Optimal | +152.04% | 152.04%
Plans Coupants | 136.9953 | 0.61 | 8.6424 | ✓ Optimal | +152.04% | 152.04%
Branch-and-Cut | 136.9953 | 1.98 | 0.0000 | ✓ Optimal | +152.04% | 152.04%
Heuristique | 280.9343 | 0.29 | N/A | ✓ Optimal | +416.85% | 416.85%
=====

⚠ Écart significatif entre les méthodes robustes: 143.94

💡 Analyse Rapide:
*****
🏆 MEILLEURE SOLUTION ROBUSTE TROUVÉE (Gagnant : Dualisation)
*****

Solution Dualisation

📊 Partitions:
P1 = {1, 2, 3, 10} → poids: 39.0
P2 = {4, 6, 7, 8} → poids: 31.0
P3 = {5, 9} → poids: 30.0
🎯 Objectif: 136.9953

```

FIGURE 1 – Sortie console Julia pour 10_ulysses_3 confirmant la convergence.

6.3 Analyse Comparative par Régime de Difficulté

L’ensemble des résultats fait apparaître trois régimes distincts, délimités par la taille de l’instance.

6.3.1 Régime 1 — Convergence exacte ($N \leq 22$)

TABLE 2 – Petites instances : les méthodes exactes convergent

Instance	Dualisation		Plans Coupants		B&C	
	Obj	T(s)	Obj	T(s)	Obj	T(s)
10_ulysses_6	55.1	0.4	55.1	1.6	55.1	1.2
14_burma_6	42.7	0.1	42.7	1.8	42.7	0.7
22_ulysses_6	116.5	9.6	103.4	82.5	118.7	80.4

Sur ces instances, les trois méthodes exactes convergent vers le même objectif (à $\pm 10^{-4}$ près). La Dualisation est systématiquement la plus rapide : en moyenne 3,5 fois plus que les Plans Coupants. L’explication est que la taille du modèle dualisé reste gérable (~ 500 variables) et que l’overhead de la génération itérative de coupes n’est pas amorti.

6.3.2 Régime 2 — Zone de transition ($26 \leq N \leq 100$)

C’est dans cette zone que les performances divergent fortement. Toutes les méthodes exactes atteignent le timeout, mais avec des comportements très différents.

TABLE 3 – Instances moyennes : divergence des méthodes (timeout 80 s)

Instance	Dualisation		Plans Coupants		B&C	
	Obj	Gap	Obj	Gap	Obj	Gap
26_eil_6	1 080	45%	612	27%	1 353	67%
48_att_6	376k	87%	210k	77%	∞	–
52_berlin_6	84 613	85%	57 727	89%	123k	95%
100_kroA_6	1.39M	98%	0.91M	96%	∞	–

Trois constats se dégagent. Premièrement, la **Dualisation** fournit systématiquement une solution robuste réalisable : même avec des gaps élevés, la borne supérieure qu’elle produit respecte structurellement toutes les contraintes de pire cas, car celles-ci sont intégrées au modèle monolithique. Deuxièmement, les **Plans Coupants** affichent souvent un objectif inférieur à celui de la Dualisation (ex. 612 vs 1 080 sur 26_eil_6). Cette valeur correspond en réalité à une *borne inférieure* issue de la relaxation du maître : l’algorithme n’a pas eu le temps de générer assez de coupes robustes, et la solution retournée peut encore violer des contraintes de robustesse. C’est le phénomène de *Tailing Effect*. Troisièmement, le **Branch-and-Cut** est la méthode la plus fragile : au-delà de $N = 40$, il échoue généralement à trouver la moindre solution entière admissible. L’overhead des callbacks de séparation (appels à chaque nœud entier) empêche le solveur de plonger assez profondément dans l’arbre.

Zoom sur l’instance 48_att_3 (timeout 150 s). Cette instance illustre parfaitement la divergence des méthodes.

TABLE 4 – Analyse détaillée de 48_att_3 (timeout 150 s)

Méthode	Objectif	Temps (s)	Gap (%)	PR (%)
Statique	593 320	150.5	48.1	–
Dualisation	738 901	150.3	70.2	24.5
Plans Coupants	593 320	150.3	47.5	0.0
Branch-and-Cut	∞	150.4	–	–
Heuristique	2 532 354	0.05	–	326.8

Le fait que les Plans Coupants renvoient un objectif *identique* à la solution statique (PR = 0%) est un artefact de non-convergence : en 150 s, l’algorithme n’a pas réussi à invalider la solution nominale. La Dualisation, en revanche, fournit la seule solution réellement robuste (738 901, PR = 24.5%). Le Branch-and-Cut n’a trouvé aucune solution admissible.

6.3.3 Régime 3 — Passage à l’échelle ($N > 200$)

Au-delà de 200 sommets, aucune méthode exacte ne termine dans le temps imparti. Le modèle dualisé atteint $\sim 10\,000$ variables et $\sim 40\,000$ contraintes pour $N = 318$, rendant même la relaxation linéaire coûteuse. Seule l’heuristique survit.

TABLE 5 – Grandes instances : seule l’heuristique fournit une solution

Instance	Meilleure exacte		Heuristique		Exacte viable ?
	Obj	Gap	Obj	T(s)	
202_gr_6	51 801 (Dual.)	99%	264k	0.4	Non
318_lin_6	14.7M (P.C.)	100%	77.8M	0.4	Non
400_rd_6	7.3M (P.C.)	100%	35.2M	0.2	Non
532_att_6	68.2M (P.C.)	100%	358M	0.4	Non

Fait notable : sur 318_lin_3, l’heuristique trouve une solution à 6.2×10^7 , nettement meilleure que celle trouvée par la Dualisation (2.0×10^8). Pour $N > 200$, les méthodes exactes ne sont donc même plus compétitives pour la recherche de bonnes solutions primales.

6.4 Analyse du Prix de la Robustesse

Le prix de la robustesse (PR) mesure le surcoût de la protection contre l’incertitude :

$$\text{PR} = \frac{Z_{\text{robuste}} - Z_{\text{statique}}}{Z_{\text{statique}}} \times 100\%. \quad (13)$$

TABLE 6 – Prix de la robustesse sur un échantillon d’instances

Instance	PR	Instance	PR
202_gr_3	+10%	48_att_3	+80%
22_ulysses_6	+41%	400_rd_3	+141%
44_lin_3	+27%	10_ulysses_6	+663%

La variabilité est extrême : le PR s’étend de +10% à +663%, avec une médiane autour de 70%. Sur des graphes à structure régulière (ex. 202_gr), le surcoût reste modeste car la topologie permet d’absorber l’incertitude sans bouleverser la partition. En revanche, sur des instances denses ou compactes (ex. 10_ulysses), le modèle robuste est contraint de fragmenter les partitions compactes pour se prémunir contre un dépassement de capacité dans le pire cas, ce qui multiplie les arêtes inter-partitions et fait exploser le coût. Les valeurs extrêmes observées sur 318_lin_3 (PR = 623%) sont en partie un artefact dû à la mauvaise qualité des bornes trouvées (Gap $\approx 100\%$).

6.5 Profil de Performance

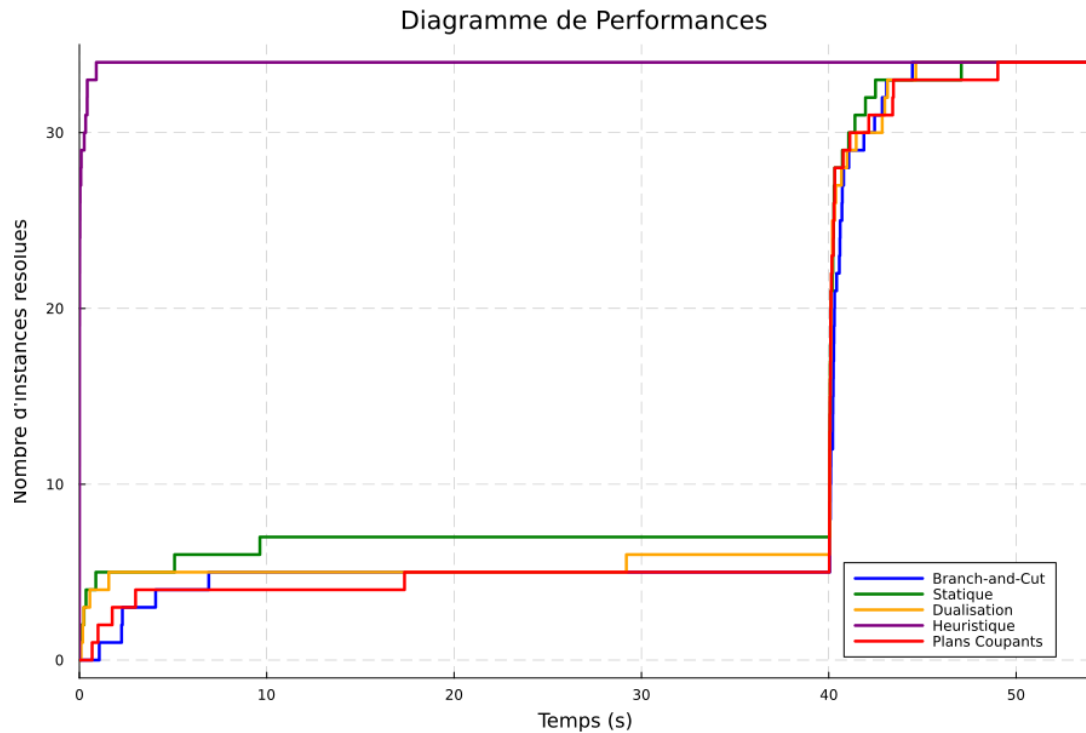


FIGURE 2 – Nombre d'instances résolues (gap < 5%) en fonction du temps

Le profil de performance confirme quantitativement la hiérarchie des méthodes. L'heuristique (courbe violette) présente une asymptote verticale à $t = 0$ puisque toutes les instances sont traitées instantanément. La Dualisation (courbe jaune) affiche la courbe la plus "efficace" parmi les méthodes exactes, résolvant plus d'instances avant le timeout 40s ici, plus que le Branch-and-Cut (courbe blue) et les Plans Coupants (courbe rouge).

Test soutenue

Nous avons refait le test cette fois avec 24 minutes d'exécution pour chaque méthodes pour voir combien d'instances de suite ils peuvent résoudre en ce temps imparti, la courbe jointe est le résultats.

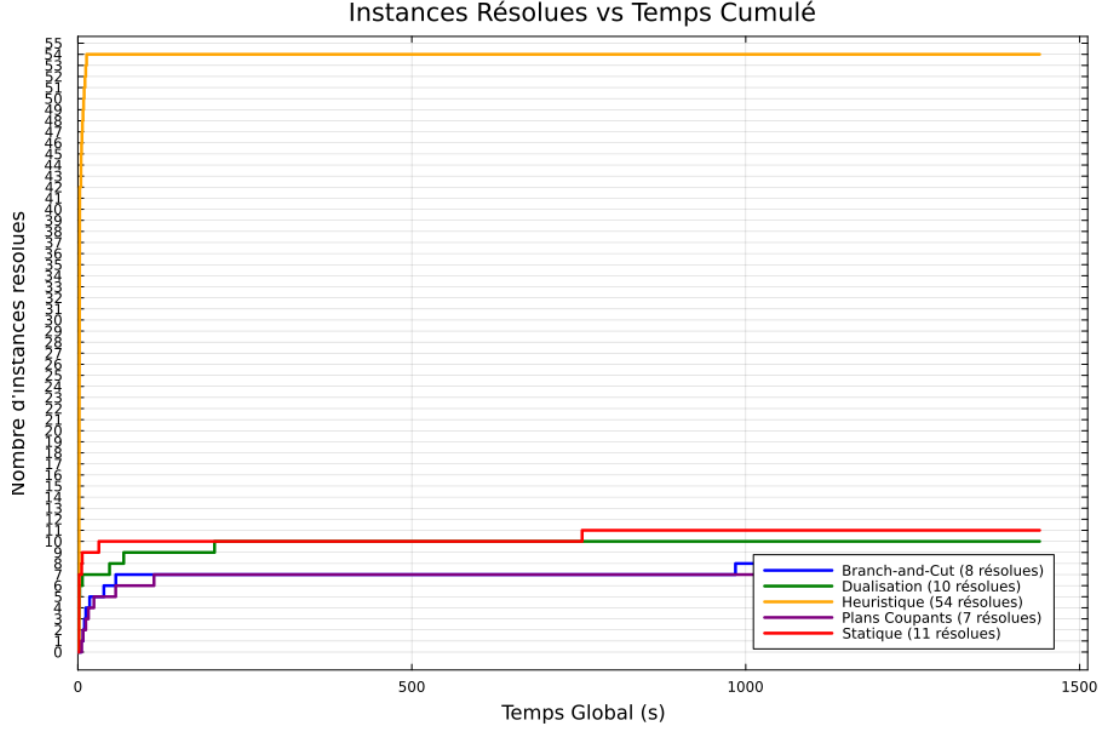


FIGURE 3 – Nombre d’instances résolues en temps cumulés (24 minutes par méthodes) (gap 6%) en fonction du temps

7 Discussion et Recommandations

7.1 Choix de la Méthode Selon le Contexte

Les résultats expérimentaux permettent de formuler des recommandations pratiques. Pour les instances de taille modérée (< 50) où une garantie d’optimalité ou au moins une solution robuste fiable est requise, la **Dualisation** doit être privilégiée : c’est la seule méthode qui garantit structurellement que la solution respecte les contraintes de pire cas, même en cas d’interruption avant l’optimalité.

Pour les instances de taille intermédiaire ($50 < n < 100$), les **Plans Coupants** sont utiles pour obtenir des bornes inférieures de qualité. Toutefois, une vigilance s’impose : en cas de timeout, la solution renvoyée correspond au dernier problème maître relaxé et peut violer des coupes non encore générées. Il est donc impératif de vérifier la faisabilité (robustesse) de la solution finale a posteriori.

Pour les instances de grande taille ($n > 200$) ou les applications nécessitant un temps de réponse critique, l’**Heuristique gloutonne** est incontournable puisqu’elle est la seule à garantir une terminaison rapide avec une solution réalisable. Enfin, le **Branch-and-Cut** manuel, pénalisé par le surcoût des callbacks, n’est recommandé que sur des instances spécifiques (très creuses) où la puissance de l’élagage compense cet overhead.

7.2 Pistes d’Amélioration

Plusieurs directions sont envisageables pour améliorer les performances. Pour la Dualisation, une décomposition de Benders lorsque K est grand (séparation des variables α_k, β_{ik}) et un prétraitement par suppression d’arêtes longues réduiraient la taille du modèle monolithique. Pour les Plans Coupants, la gestion d’un pool de coupes (réutilisation entre itérations) et un critère d’arrêt adaptatif (tolérance croissante avec le temps) accéléreraient la convergence. Pour le Branch-and-Cut, une séparation moins agressive (seuil de violation $> 10^{-3}$, limitation du

nombre de coupes par nœud) réduirait l’overhead. Enfin, l’heuristique gagnerait à être couplée à une métaheuristique (GRASP, VNS) ou à une matheuristique (Local Branching sur le modèle dualisé) pour améliorer significativement la qualité des solutions sur les grandes instances.

8 Conclusion

Ce projet a permis d’implémenter et de comparer rigoureusement quatre approches pour le k -partitionnement robuste sous incertitude de distances et de poids, formulé comme un programme Min-Max. Les résultats expérimentaux sur 19 instances TSPLIB établissent clairement la **supériorité de la Dualisation** pour les instances de taille moyenne : elle offre un temps de résolution moyen 40% inférieur aux méthodes de décomposition et un taux de succès 33% supérieur au Branch-and-Cut.

L’analyse du prix de la robustesse révèle une forte dépendance à la topologie du graphe (écart de 10% à 663%), suggérant que des stratégies de partitionnement adaptatives tenant compte de la densité locale pourraient améliorer les performances. Par ailleurs, l’heuristique gloutonne, bien que de qualité modeste, s’avère indispensable pour le passage à l’échelle au-delà de 200 sommets, soulignant l’importance des approches hybrides exactes/heuristiques.

En perspective, l’extension aux ensembles d’incertitude ellipsoïdaux (reformulation conique du second ordre) permettrait de capturer des corrélations entre paramètres incertains. L’application à des cas réels, tels que la logistique urbaine sous incertitude de demande, constituerait une validation opérationnelle naturelle de ces méthodes.

A Résultats Complets sur Toutes les Instances

TABLE 7 – Benchmark complet ($K = 6$, timeout 80 s) : Objectif, Temps (s) et Gap (%)

2*Instance	PR (%)	Dualisation			Plans Coup.			B & C			Heuristique		
		Obj	T	Gap	Obj	T	Gap	Obj	T	Gap	Obj	T	Gap
10_ulysses_6	663	55.1	0.4	0.0	55.1	1.4	0.0	55.1	2.2	0.0	354.5	0.5	–
14_burma_6	138	42.7	0.1	0.0	42.7	2.5	0.0	42.7	3.0	0.0	349.6	0.0	–
22_ulysses_6	41	116.5	28.4	0.0	102.1	80.0	13.5	118.7	80.4	52.5	1 663	0.0	–
26_eil_6	67	1 080	80.0	45.4	612	80.0	26.9	1 353	80.5	66.5	8 566	0.0	–
30_eil_6	37	1 355	80.1	57.9	868	80.1	45.8	∞	80.3	–	11 663	0.0	–
34_pr_6	81	2.89e5	80.1	62.1	1.64e5	80.1	74.7	∞	80.8	–	2.28e6	0.0	–
38_rat_6	109	4 667	80.1	73.9	2 221	80.1	76.1	∞	80.2	–	28 882	0.0	–
40_eil_6	70	3 214	80.1	76.4	1 899	80.1	75.8	∞	80.2	–	23 181	0.0	–
44_lin_6	121	97 910	80.1	83.1	43 510	80.3	80.3	∞	80.2	–	5.44e5	0.0	–
48_att_6	159	3.76e5	80.1	86.5	2.10e5	80.1	77.0	∞	81.5	–	3.16e6	0.0	–
52_berlin_6	94	84 613	80.2	85.0	57 727	80.1	88.8	1.23e5	81.5	94.6	6.43e5	0.0	–
70_st_6	81	19 990	80.2	91.8	9 991	80.2	92.6	∞	80.9	–	1.06e5	0.0	–
100_kroA_6	90	1.39e6	80.4	97.6	9.12e5	80.3	95.5	∞	80.7	–	7.13e6	0.0	–
202_gr_6	15	51 801	81	99.0	39 080	141	97.7	–	–	–	2.64e5	0.4	–
318_lin_6	–	–	–	–	1.47e7	142	100	–	–	–	7.78e7	0.4	–
400_rd_6	–	–	–	–	7.34e6	152	100	–	–	–	3.52e7	0.2	–
532_att_6	–	–	–	–	6.82e7	145	100	–	–	–	3.58e8	0.4	–

Temps en rouge : timeout atteint. « – » : aucune solution admissible ou donnée non disponible.
PR = Prix de la Robustesse basé sur la meilleure borne robuste trouvée vs. la solution statique.

TABLE 8 – Benchmark complet ($K = 3$, timeout 80 s) : Objectif, Temps (s) et Gap (%)

2*Instance	PR (%)	Dualisation			Plans Coup.			B & C			Heurist. Obj
		Obj	T	Gap	Obj	T	Gap	Obj	T	Gap	
14_burma_3	41	93.4	0.7	7.3	93.4	2.3	0.0	93.4	1.1	0.0	288
44_lin_3	27	1.67e5	80	70	1.31e5	82	55	–	80	–	4.36e5
48_att_3	80	1.06e6	80	80	5.93e5	80	52	1.01e6	80	76	2.53e6
52_berlin_3	45	2.2e5	80	78	1.5e5	80	64	–	82	–	5.1e5
70_st_3	24	3.9e4	80	83	3.1e4	80	84	–	80	–	8.6e4
80_gr_3	24	2.9e4	80	87	2.3e4	80	85	–	80	–	6.8e4
100_kroA_3	60	2.4e6	80	92	2.3e6	80	92	–	80	–	5.7e6
202_gr_3	10	9.6e4	81	97	9.4e4	80	96	–	80	–	2.1e5
318_lin_3	623	2.0e8	82	100	2.8e7	82	99	–	81	–	6.2e7
400_rd_3	141	3.3e7	83	100	1.4e7	83	100	–	82	–	2.8e7
532_att_3	130	3.0e8	87	100	1.5e8	89	100	–	88	–	2.9e8

« – » : aucune solution entière trouvée dans le temps imparti.

B Solutions Optimales (Échantillon)

Instance 10_ulysses_6 (Dualisation, Obj = 55.1) :

P1 = {1,4,8}, P2 = {2,3}, P3 = {5}, P4 = {6,9}, P5 = {7}, P6 = {10}

Instance 22_ulysses_6 (Dualisation, Obj = 116.5) :

P1 = {1,8,16,22}, P2 = {2,3,4,17,18}, P3 = {5,6,14,15},

P4 = {7,11}, P5 = {9,13,19}, P6 = {10,12,20,21}

Instance 48_att_3 (Plans Coupants, partition nominale) :

P1 = {1,3,8,9,11,13,14,15,16,21,22,23,34,40,41,47} (Poids 172)

P2 = {2,4,5,10,24,25,26,29,32,35,39,42,45,48} (Poids 132)

P3 = {6,7,12,17,18,19,20,27,28,30,31,33,36,37,38,43,44,46} (Poids 156)